

Hardware required for building an embedded application on Qt is as follows:

1. Raspberry Pi 3 board
2. Touchscreen or computer monitor
3. USB keyboard and mouse
4. Memory card (16GB or 8GB)
5. Power adaptor (5V, 2A)

## Preparing Raspberry Pi for Qt framework

Download Raspbian OS from the Internet, and install it on the SD card (NOOBS is recommended).

Internet connectivity is required for Raspberry Pi. Open the terminal on Raspberry Pi by pressing ctrl + alt + t simultaneously on the keyboard. Then, check GCC compiler. If it is not available, install it using the following command:

```
$ Sudo apt-get install gcc
```

Use the following commands to

install Qt Creator:

```
$ sudo apt-get update
sudo apt-get upgrade
sudo apt-get install qt4-dev-tools
sudo apt-get install qtcreator
sudo apt-get install git-core
sudo apt-get install subversion
```

After successful execution of these steps, Qt Creator will be added to the programming list on Raspberry Pi OS on the desktop.

## Setting up Qt platform on Raspbian OS

When running Qt Creator for the first time, configure the following settings:

1. Go to Tools > Options > Build & Run > Compilers. Compilers window is shown in Fig. 1.
2. Click on Add > GCC. Set the path to usr/bin/arm-linux-gnueabi-hf-gcc-4.9, as shown in Fig. 2. Then, click on Apply to save changes.

3. Go to Qt Versions > Add > Browse (usr/bin/qmake-qt4) and select the latest version, as shown in Fig. 3. Click on Apply.

4. Go to Kits > Add. Set Device Name as Raspberry Pi (or any other name of your choice). Select Device Type as Desktop, and Device as Local PC, as shown in Fig. 4. Click on Apply.

## Project development with Qt

You are now ready to begin with the first project. The steps are as follows:

1. Click on New Project.
2. Select Applications > Qt Widgets Application > Choose..., as shown in Fig. 5.
3. Type in the name of your project (HelloWorld) and choose the destination folder where you wish to save your project, as shown in Fig. 6.
4. Select the kit as Raspberry Pi (or the name that you chose in Fig. 4 above for your kit) and click on Next, as shown in Fig. 7.

5. Continue with default names (MainWindow class, mainwindow.h, mainwindow.

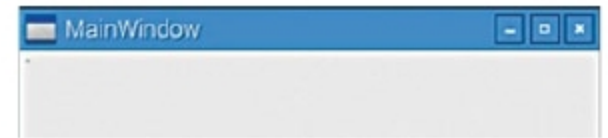


Fig. 10: MainWindow screen

```
1 #include "mainwindow.h"
2 #include <QApplication>
3 #include <QLabel>
4
5 int main(int argc, char *argv[])
6 {
7     QApplication a(argc, argv);
8     MainWindow w;
9     QLabel *label=new QLabel(&w);
10    label->setText("Hello World");
11    label->setAlignment(Qt::AlignCenter);
12
13    w.show();
14
15    return a.exec();
16 }
17
```

Fig. 11: Default main.cpp screen



Fig. 12: Hello World screen

cpp) in the class information sub-window, as shown in Fig. 8. Click on Next.

6. All files to be added in the project should be listed here. Click on Finish.

A new project called HelloWorld will be created. Project Explorer will show all relevant project files, as shown in Fig. 9.

7. Click on Build > Build Project. The console window should show no errors if the kit is correctly configured.

8. Click on Build > Run. MainWindow form will be displayed, as shown in Fig. 10.

9. When you click on main.cpp, the default screen will appear.

10. Add the following code to main.cpp to display 'Hello World' on the main window form as shown in Fig. 11.

```
#include <qlabel.h>
QLabel *label=new QLabel (&w);
label->setText("Hello World");
label->setAlignment(Qt::AlignCenter);
```

11. Click on Build > Run. The form with 'Hello World' on the screen will be displayed, as shown in Fig. 12. **EFY**

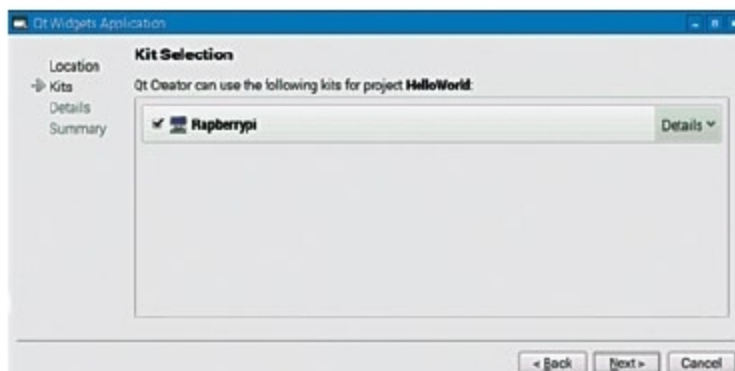


Fig. 7: Select Raspberry Pi kit

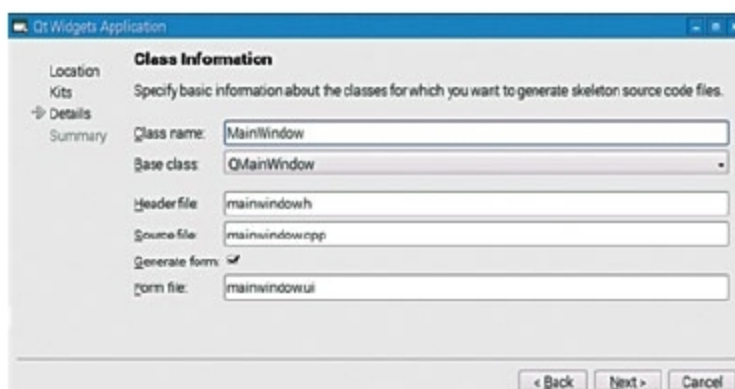


Fig. 8: Class information window

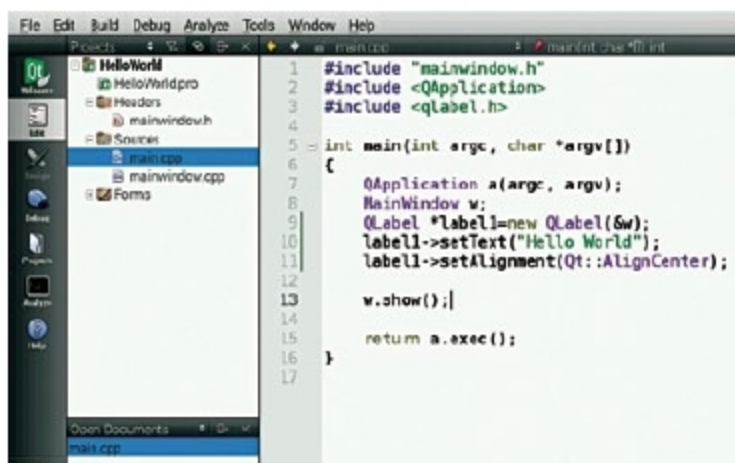


Fig. 9: HelloWorld project on Project Explorer window

